

AMATH 482/582: HOME WORK 5

EUNICE SEUNG-HYUN HAN

Department of Applied Mathematics, University of Washington, Seattle, WA
eunich@uw.edu

ABSTRACT. This study classifies images in the FashionMNIST data set. Results indicate how effectively Fully Connected Deep Neural Networks and Convolutional Neural Networks with hyperparameter tuning and applying configurations were able to present high performance in classifying the fashion items of 10 classes. Also, setting the budget of weights for both networks would allow us to compare how each classification performs. Tables show how the change of configurations impacts the accuracy of each network's model. This study can be used in the Information Technology and the Artificial Intelligence industry as the classifier algorithm that predicts the fashion item by categories could be used for enhanced search in the fashion industry.

1. Introduction and Overview

There are a lot of different kinds of fashion items, including shirts, pants, and shoes. This report aims to classify images of different fashion items from the FashionMNIST data set by applying the Fully Connected Deep Neural Networks (FCNs) and the Convolutional Neural Networks (CNNs) as the classifiers to categorize each image into one of the 10 clothing classes. The FashionMNIST data set consists of 60K training images and 10K testing images while each sample is in grayscale 28×28 image. Through designing the FCN and CNN model with hyperparameter tuning and using the training loss curve and validation accuracy, the most accurate model would be applied to the test set with high prediction. By implementing the trained FCN and CNN model for different budget of weights, the report demonstrates which model with which particular parameter value would show the highest accuracy in prediction for classifying images of different fashion items and would be able to configure the significant difference incorporating the neural network under the budget of weights, allowing to achieve the most accurate model with reasonable computation time and high accuracy.

2. Theoretical Background

The FCN (Fully Connected Network) model is built with stacking multiple layers of neurons. When layer ℓ has n_ℓ neurons, then each of those neurons are connected to $n_{\ell-1}$ neurons from the previous layer, $(\ell - 1)$. Each layer transforms its input vector which is from the previous layer into an output vector which will eventually be the input of the next layer. The network starts with the raw data as the input layer which will flow through the hidden layers, and end with the output layer which provides the probability score distribution in classification tasks. Each layer calculates the output vector which will be the input vector for the next layer. When training the FCN model, the dataset is split into smaller sets which is the batches (selected manually). Every batch would follow the steps below through all layers. After all the batches from the entire dataset is trained, we call that one epoch is completely passed. With the selected number of epoch, the model continues training for the most accurate and high prediction.

Forward pass: — Every batch goes through the layers as below —

- (1)
$$z = \vec{w}^T \vec{x} + b$$
- (2)
$$\hat{y} = \sigma(z) \implies L(\hat{y}, y)$$
 - $\vec{x} \in \mathbb{R}^n$ (input vector) $\vec{w} \in \mathbb{R}^n$ (weight vector) $b \in \mathbb{R}$ (bias vector)
 - $\sigma()$ is the activation function which is ReLU activation in this assignment; $\text{ReLU}(x) = \max(0, x)$

The forward pass computes the logits z in Equation (1) and then applies the ReLU activation function in Equation (2) for every layer in the FCN model. As this process repeats for all layers, the final layer ℓ , is the prediction of this network/model which provides class probabilities.

— To get the weight vector $\vec{w}$ and bias b the following equations are needed —

- (3) Cross Entropy Loss:
$$L(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

- (4) Back propagation:
$$\nabla_{\vec{w}} L(\hat{y}, y) = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \nabla_{\vec{w}} z$$

Cross Entropy Loss measure how different the predicted distribution of the model is from the true distribution/labels. By minimizing the cross entropy, it maximizes the likelihood of the prediction of the model.

Then the backpropagation computes the gradient of the loss function with respect of each weight. After computing backpropagation, we need to update the parameters such as the weight and bias of the model to reduce the loss of the model and maximize the model's prediction which is the optimizers that does that.

Convolutional Neural Networks (CNN) is a neural network that uses convolutional layer to extract features and its local patterns from the grid-like data and uses learnable kernels that convolve across the grid-like data (images). After the kernel is learned in the training process, the same kernel moves to the every next position of the input data in the forward pass(1) which forms the output map with captured feature and local patterns of every positions it convolve. As the same kernel is trained for the local patterns of the image by positions, this improves detections of features in the corners. This process allows to detect the patterns and feature in early layers and combines them so it could abstract more in details in deeper layers. Stepping into the Convolutional Neural Network pipeline process, the input image that is 28×28 , goes into the first convolution layer.

$$(5) \quad \text{Conv2D}(k)[i, j] = \sum_{c=0}^{C-1} \sum_{u=0}^{f-1} \sum_{v=0}^{f-1} X(c)[i+u, j+v] W_k(c)[u, v] + b_k$$

- $X(c)$: Input for channel c $W_k(c)$: Kernel with output channel k & input channel c b_k : Bias

Equation (5) shows how the convolutional layer receives the output feature maps from passing the grayscale of each input image. The sum for u and v adds up the input, $X(c)[i+u, j+v]$, multiplied with its corresponding kernel/weight $W_k(c)[u, v]$. $\text{Conv2D}(k)[i, j]$ demonstrates the output feature maps of k channels from the input channel c . In k feature maps, it presents and distinguishes different information such as the features or components of the input image. Then the Pooling layer (Max Pool) reduces the spatial size of the output feature map, $\text{Conv2D}(k)[i, j]$, such as,

$$(6) \quad \text{Pool}(k)[p, q] = \max_{\substack{0 \leq u < 2 \\ 0 \leq v < 2}} (\text{ReLU}(\text{Conv2D}(k))[2p+u, 2q+v])$$

After pooling, the input size would go from $28 \times 28 \rightarrow 14 \times 14$ in here and these steps repeat on the second convolution layer with the updated input channels and kernel. The pooling would reduce the time for computation and control overfitting as it focuses on generalization. After the final pooling layer, the pooled feature map flattens into a 1D vector. And this vector would go through the fully connected layers,

$$(7) \quad \text{Linear Transformation: } z_\ell = W_\ell \vec{x}_{\ell-1} + \vec{b}_\ell$$

$$(8) \quad \text{ReLU Activation: } \vec{x}_\ell = \max(0, z_\ell)$$

$$(9) \quad \text{Final layer: } z_{\text{final}} = W_{\text{final}} \vec{x}_{\text{last hidden}} + \vec{b}_{\text{final}}$$

The z_{final} would produce the final output, the classification score, for predicting the clothing classes.

— Optimizers —

SGD (Stochastic Gradient Descent) [5] computes the gradients of the loss function, which is the cross entropy loss function in this assignment, with respect to the weight and bias (parameter of the model) on the batches. SGD Optimizer (when $J(\vec{w}, b) = L(\hat{y}(\vec{w}, b), y)$):

$$(10) \quad \vec{w} \leftarrow \vec{w} - \alpha \nabla_{\vec{w}} J(\vec{w}, b), \quad b \leftarrow b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

- $\nabla_{\vec{w}} J(\vec{w}, b)$ is the gradient of J with respect to $\vec{w}$
- $\frac{\partial}{\partial b} J(\vec{w}, b)$ is the derivative of J with respect to bias, b
- α : learning rate that controls the size of the step in the direction of the negative gradient by time

The SGD updates the parameters to reduce the loss through many batches and epochs, it effectively finds the accurate parameters for the model for better prediction. As it uses small datasets, batches, instead of the entire dataset every time, it is less time consuming but still provides high accuracy. RMSProp (Root Mean Squared Propagation) [5] improves SGD by adapting the learning rate for each parameter based on the gradients of the previous history. RMSProp Optimizer:

$$(11) \quad r_{\vec{w}} \leftarrow \beta r_{\vec{w}} + (1 - \beta)(\nabla_{\vec{w}} J(\vec{w}, b))^2, \quad \vec{w} \leftarrow \vec{w} - \frac{\alpha}{\sqrt{r_{\vec{w}} + \varepsilon}} \nabla_{\vec{w}} J(\vec{w}, b)$$

$$(12) \quad r_b \leftarrow \beta r_b + (1 - \beta)\left(\frac{\partial}{\partial b} J(\vec{w}, b)\right)^2, \quad b \leftarrow b - \frac{\alpha}{\sqrt{r_b + \varepsilon}} \frac{\partial}{\partial b} J(\vec{w}, b)$$

- β is the exponential decay factor that the default is 0.9 in RMSProp
- ε is a small positive constant number that prevents division by 0
- Adapted learning rate of $\vec{w}$: $\frac{\alpha}{\sqrt{r_{\vec{w}} + \varepsilon}}$ Adapted rate of bias, b : $\frac{\alpha}{\sqrt{r_b + \varepsilon}}$

RMSProp normalizes each parameter step size which prevents the updates to not be aggressive. As this keeps track on the gradient history and uses adaptive learning rates, it is faster and stable than SGD. Adaptive Moment Estimation[5] combines averaging the gradients directly ($m_{\vec{w}}$ and m_b) and the squared gradients ($r_{\vec{w}}$ and r_b) which has smoother gradients updates with adaptive steps for parameters. Adam Optimizer:

$$(13) \quad m_{\vec{w}} \leftarrow \beta_1 m_{\vec{w}} + (1 - \beta_1) \nabla_{\vec{w}} J(\vec{w}, b), \quad r_{\vec{w}} \leftarrow \beta_2 r_{\vec{w}} + (1 - \beta_2) (\nabla_{\vec{w}} J(\vec{w}, b))^2$$

$$(14) \quad m_b \leftarrow \beta_1 m_b + (1 - \beta_1) \frac{\partial}{\partial b} J(\vec{w}, b), \quad r_b \leftarrow \beta_2 r_b + (1 - \beta_2) \left(\frac{\partial}{\partial b} J(\vec{w}, b) \right)^2$$

Adam Optimizer first updates the first moment estimates for both weight vector and bias as in Equations (13, 14). Then update the second moment estimates by averaging the squared gradients for both weight vector and bias. Initially $m_{\vec{w}}$ and m_b can be biased toward zero and to prevent this, Adam computes:

$$(15) \quad \hat{m}_{\vec{w}} = \frac{m_{\vec{w}}}{1 - \beta_1^t}, \quad \hat{m}_b = \frac{m_b}{1 - \beta_1^t}, \quad \hat{r}_{\vec{w}} = \frac{r_{\vec{w}}}{1 - \beta_2^t}, \quad \hat{r}_b = \frac{r_b}{1 - \beta_2^t} \quad t : \text{step count of batch iterations}$$

Then the final parameter update for the Adam optimizer looks like:

$$(16) \quad \vec{w} \leftarrow \vec{w} - \frac{\alpha \hat{m}_{\vec{w}}}{\sqrt{\hat{r}_{\vec{w}} + \epsilon}}, \quad b \leftarrow b - \frac{\alpha \hat{m}_b}{\sqrt{\hat{r}_b + \epsilon}}$$

Adam optimizer helps prevent the small local minima and smooths the noisy gradients which adjusting the parameters' effective step size based on the information of the gradients. Adam combines momentum and RMSProp so it converges faster and adapts the learning rate for each weight and bias. FashionMNIST dataset contains nonlinear data so finding the optimizer through hyperparameter tunings in [Task 3](#) would effectively help to determine which optimizer performs the model to have better prediction. Training Loss is the average of the loss function from Equation (3), L , over all training samples.

$$(17) \quad \text{Training Loss} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} L(\hat{y}^{(i)}, y^{(i)})$$

The validation accuracy is the the fraction of validation sample the model classifies correctly. When the model outputs the probability distribution, $\hat{p}^{(i)}$, the predicted class is selected by:

$$(18) \quad \hat{c}^{(i)} = \arg \max_c \hat{p}_c^{(i)} \quad \text{Validation Accuracy} = \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} 1(\hat{c}^{(i)} = y^{(i)})$$

The indicator function, $1(\hat{c}^{(i)} = y^{(i)})$, tells when the predicted class correctly matches the true label is gives a 1 and 0 otherwise. The higher the validation accuracy is, it means there are more correctly classified validation samples. The training loss (17) and validation accuracy (18) allows to identify if the model is overfitting or underfitting which is effective in monitoring if the model is accurate. When the training metrics improves while the validation metric gets worse we can assume the model is overfitting which tells us to adjust the hyperparameters such as the learning rate. Dropout regularization [5] randomly drops a fraction of neurons from layers when training the model which helps to prevent overfitting of the training. The dropout at layer ℓ looks like:

$$(19) \quad \vec{a}_{\ell}^d = \vec{a}_{\ell} \times \frac{\vec{d}_{\ell}}{1 - p_d}$$

- $\vec{a}_{\ell}$ is the activation vector at layer ℓ $\vec{d}_{\ell}$ is the dropout mask with each entry either 1 or 0

Initialization [5] is used to set the initial values of the network's parameters such as weights and biases before the training starts. This helps prevent vanishing or exploding gradients and provide stable learning and faster convergence. Random Normal Initialization [5] uses normal distribution, $w \sim \mathcal{N}(\mu, \sigma^2)$. This is a simple technique in terms of deep learning but if σ is too large or small, it may lead to exploding or vanishing activations. Xavier Normal Initialization [5] is used for networks with tanh or sigmoid activations. It keeps the variance of the activations across layers. The Xavier Normal equation for each weight is, $w \sim \mathcal{N}(0, \frac{2}{n_{\text{in}} + n_{\text{out}}})$ when n_{in} is the number of input neurons and n_{out} is the number of output neurons of the layer. This technique works well in tanh or sigmoid networks and reduces the risk of vanishing or exploding gradients compared to normal initialization. Kaiming (He) Uniform Initialization [5] was proposed for ReLU activations as ReLU needs a slightly different variance as it zeros out the negative inputs. Kaiming Uniform is $w \sim \text{Uniform}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right)$. As the ReLU sets the negative inputs as zero, which halves the active neurons effectively and this technique helps to prevent vanishing and exploding in ReLU based networks. Batch Normalization [5] normalizes the activations in each layer per each batches so each neuron has roughly mean as zero with unit variance but still have network to contain the overall scale which reduces the sensitivity of hyperparameters. Batch Normalization first computes the mean of each batch by, $\mu_B = \frac{1}{B} \sum_{i=1}^B z_i$. Then

it computes the variance, $\sigma_B^2 = \frac{1}{B} \sum_{i=1}^B (z_i - \mu_B)^2$. Using this mean and variance, the normalization and rescale works by, $\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, $y_i = \gamma \hat{z}_i + \beta$ where y_i is the next layer's input.

Calculating weights in the design of the Fully Connected Network (FCN) model:

$$(20) \quad (I \times h) + h + (h \times O) + O \leq B_w$$

• I : Size of output layer h : Size of hidden layer O : Size of output layer

Calculating weights in the design of the Convolutional Neural Networks (CNN) model:

$$(21) \quad \underbrace{(K_h \cdot K_w \cdot C_{in} \cdot C_{out} + C_{out})}_{\text{Convolution}} + \underbrace{((H \cdot W \cdot C_{out}) \cdot h + h)}_{\text{FC hidden}} + \underbrace{(h \cdot C_{out,final} + C_{out,final})}_{\text{FC output}} \leq B_w$$

- K_h, K_w : Convolution kernel height/width B_w : Budget weight
- C_{in}/C_{out} : Number of input/output channels for the Convolution layer
- H, W : Height and width of the feature maps after (or within) the convolution layer
- h : Size of the fully connected hidden layer $C_{out,final}$: Number of output neurons in the final layer

3. Algorithm Implementation and Development

The algorithms and techniques that were used are CNN, FCN [5] using Cross Entropy loss (3), optimizers (RMSProp (2), Adam (16), SGD (10)), dropout regularization (19), initializations (Random Normal, Xavier Normal, Kaiming (He) Uniform), and batch normalization. The Fully Connected Neural Network composes two functions. The first function initializes the model by constructing the network layers and applying activation functions. The for loop in the first function iterates over the list of hidden layers, appending each layer and connecting it to the next with a ReLU activation [5] and then adds the output layer. The second function defines the forward pass (1) and flattens the input which goes through the network to generate the logits of each class (fashion item). Then there is another function that initializes the neural network with specific input, output, and hidden layer dimensions and trains FashionMNIST data then computes the training loss (17), validation loss, validation accuracy (18).

The Convolutional Neural Networks composes two functions. The first function initializes the model by constructing the network layers with defining the convolutional features. First the input passes the convolutional layers (5) and apply the activation function and then pools (6) the layers which reduce the size of the input image. Then the for-loop in the first function iterates over the list of hidden layers, appending each layer and connecting it to the next with a ReLU activation [5] and then adds the output layer, creating a fully connected network. The second function defines how the model propagates the input through the network by reshaping the input size if it is flattened. If the input size is flattened, the function reshapes it to be [batch size, number of channels, 28, 28] as each image sample has 28×28 in grayscale, which goes through the network to generate the features of each class (fashion item).

Then there is another function that initializes the neural network with specific input, output, and hidden layer dimensions and trains FashionMNIST data then computes the training loss (17), validation loss, validation accuracy (18) by each the FCN and CNN model with its parameters. The function starts with the outer for-loop that trains the FCN and CNN model built ahead with the selected parameters/configurations. For FCN the specific parameters would be the number of neurons for the first, hidden, last layer and the optimizer, learning rate, and epoch. And for CNN the specific parameters would be the number of neurons for the first, hidden, last layer, number of output channels for the convolution layers, kernel size, and the optimizer, learning rate, and epoch. The outer for-loop that runs through each epoch has an inner for-loop that processes the training batches by first setting the FCN/CNN model into training mode and resetting the gradients. Then the model would get predictions using forward pass 1, 2 and the loss function, cross entropy loss (3), would calculate the training loss of the model for cross validation or for such selected configurations for each given tasks. Then the back propagation (4) is applied to calculate the gradients for the optimizer to update. After processing all batches for each epoch, the training loss for each epoch is presented. Inside the same for-loop that runs for each epoch, there is another inner for-loop that evaluates the validation batches and computes the validation loss and accuracy by comparing the model's predictions with ground-truth labels in parallel to the validation labels. The validation accuracy (18) is determined by dividing the number of correct predictions by the total number of samples. And the testing accuracy function has the same structure as the for-loop that computes the validation accuracy. Each of the Tasks have different hyperparameter tunings for parameters such the optimizer type, learning rate, initialization methods, adjustments of batch normalization, and configurations by its neural networks. Since the structure of the FCN and CNN models are modular, by simply passing the parameters in the training procedure, it would provide that particular model of using specific parameters, training loss, validation loss/accuracy,

and testing accuracy. The software packages employed are NumPy [3], Matplotlib [4], Scikit-learn [6], and PyTorch [5]. NumPy [3] facilitates numerical computations such as getting the average. Matplotlib [4] provides the training loss curve and the validation accuracy curve by each epochs. Scikit-learn [6] is a machine learning library that splits the train and test data the model. PyTorch [5] is an deep learning framework that provides tools for building and training neural networks in python which includes Fully Connected Neural Network and Convolutional Neural Networks that are used to classify fashion items.

4. Computational Results

Task 1: [FCN – 100K weight]

# of neurons: 1 st layer	# of neurons: last layer	# of hidden layers & neurons	Epoch
784	10	1 & 125	20

TABLE 1. The parameters/configurations of FCN model weight $\leq 100,000$.

The number of neurons in hidden layer was selecting by calculating the weights to be under 100K in the FCN model. Equation (20) was applied $(784 \cdot h + h) + (h \cdot 10 + 10) \leq 100,000 \rightarrow h \leq 125.77$, which is why 125 neurons was selected with weight 99,385. By cross validating through multiple learning rates for optimizers,

LR Optimizer	1e-1	1e-2	1e-3	1e-4	1e-5	1e-6
RMSProp	23.31/38.9%	0.57/83.78%	0.3/87%	0.42/85.5%	0.83/77%	1.96/44.6%
Adam	1.17/62.33%	0.28/87.11%	0.29/88%	0.47/84.76%	0.94/74.6%	2/41.2%
SGD	0.37/85.7%	0.56/82.2%	1.07/71.3%	2.03/39.2%	2.3/11.9%	2.37/5.8%

TABLE 2. Training loss / Validation accuracy of Optimizers with different Learning rates and with Dropout = 1e-4.

LR Optimizer	1e-1	1e-2	1e-3	1e-4	1e-5	1e-6
RMSProp	19.2/40%	0.6/83.6%	0.3/87%	0.4/85.5%	0.82/73.9%	2.1/29.41%
Adam	1.5/58.65%	0.3/87.7%	0.28/88.4%	0.45/85.4%	0.96/70.1%	2.24/23.6%
SGD	0.36/86.77%	0.51/83.1%	0.84/74.2%	1.72/44.3%	2.92/18.3%	2.88/7%

TABLE 3. Training loss / Validation accuracy of Optimizers with different Learning rates and with Kaiming (He) Uniform Initialization.

Table 2 shows the training loss and validation accuracy of each optimizers with different learning rate with using the dropout = 1e-4 which showed a slightly high accuracy in HW4. This configuration checks if dropout regularization improves the accuracy in this FCN 100K model and Table 2 shows lower validation accuracy, which tells dropout regularization does not improve this model. Table 3 shows the training loss and validation accuracy of each optimizers with different learning rate with using Kaiming (He) Uniform Initialization. This initialization showed the best performance in HW4 but didn't show an improvement in the model, although this method works best with the ReLU activation and that's why this configuration was applied.

Training Loss	Validation Accuracy	Testing Accuracy
0.28	88.4%	88.52%

TABLE 4. Training loss, Validation and Testing accuracy.

As a result, in this FCN model with Table 1 parameters, using Adam optimizer with learning rate 1e-3 and Kaiming (He) Uniform initialization, shows that the testing classification accuracy is above 88%.

Task 2:

[FCN – 50K weight]

Hidden layers	Hidden neurons	Epoch
1	62 neurons	20

TABLE 5. The parameters/configurations of FCN model weight $\leq 50,000$.

Number of neurons in hidden layer was selected by calculating the weights to be $\leq 50K$ in FCN model. Equation (20) was applied and selected 62 neurons because $(784 \cdot h + h) + (h \cdot 10 + 10) \leq 50,000 \rightarrow h \leq 62.88$ with weight 49,300.

Learning rate	1e-1	1e-2	1e-3	1e-4	1e-5	1e-6
Adam optimizer	1.24/53.5%	0.33/87.24%	0.31/87.9%	0.5/84.4%	1.16/63.64%	2.1/30.3%

TABLE 6. Training loss/Validation accuracy of using Adam optimizer with different Learning rates and with Kaiming (He) Uniform Initialization.

Training Loss	Validation Accuracy	Testing Accuracy
0.31	87.9%	87.73%

TABLE 7. Training loss, Validation and Testing accuracy.

Based on Task 1, by using the same configuration except the number of hidden neurons, the cross validation for the accurate learning rate is applied here. Table 6 shows this FCN model with Table 5 parameters, using Adam optimizer with learning rate 1e-3 and Kaiming (He) Uniform initialization, has the highest validation accuracy. Applying this model to the test set, the testing accuracy is 87.73% as in Table 7 which performs poor than the FCN 100K model in Task 1. This may happen because it is using approximately half of the weight compared to Task 1 which means it takes fewer operations for training and has potential for the network to learn less on complex functions and features than larger weight.

[FCN – 200K weight]

Hidden layers	Hidden neurons	Epoch
1	251 neurons	20

TABLE 8. The parameters/configurations of FCN model weight $\leq 200,000$.

Neurons in hidden layer was selected the weights to be $\leq 200K$ in FCN model. Equation (20) was applied and selected 251 neurons because $(784 \cdot h + h) + (h \cdot 10 + 10) \leq 200K \rightarrow h \leq 251.56$ with weight 199,555.

Learning rate	1e-1	1e-2	1e-3	1e-4	1e-5	1e-6
Adam optimizer	1.55/62.8%	0.32/88.11%	0.25/88.63%	0.4/86.44%	0.83/73.05%	1.91/39.1%

TABLE 9. Training loss/Validation accuracy of using Adam optimizer with different Learning rates and with Kaiming (He) Uniform Initialization.

Training Loss	Validation Accuracy	Testing Accuracy.
0.25	88.63%	87.88%

TABLE 10. Training loss, Validation and Testing accuracy.

The process for this model works the same for when the budget of the weight is $\leq 200,000$ as in $\leq 50,000$. Also Table 10 shows this FCN model with Table 8 parameters, using Adam optimizer with learning rate 1e-3 and Kaiming (He) Uniform initialization, had the highest validation accuracy which is same to when the budget weight is $\leq 50,000$. Even though the number of hidden neurons are different, the overall validation and testing accuracy isn't showing a big difference between the FCN model with below 50K weights and 200K weights. Also they are both lower than the testing accuracy of the budget weight 100K from FCN 100K model Task 1 which tells changing the weights in this certain FCN model does not make a big difference in classifying the dataset. This may happen as weight of 50K may underfit and 200K may be less stable as the optimizer does not do well in higher dimensional space.

Task 3:

	Hidden layers	Hidden neurons	Output channels in 1 st & 2 nd Convolution layer	Kernal	Epoch
1 st	1	60	16 & 32	3	20
2 nd	1	40	20 & 40	3	20
3 rd	1	40	16 & 40	5	20

TABLE 11. Multiple parameters/configurations of CNN model weight $\leq 100,000$.

Table 11 shows multiple configurations for the CNN model that the weight is below 100K. The weights are computed from Equation (21). Given an example for the first configuration, the weight is 99,550 because the number of hidden neurons of the chosen output channels is 60 as $(1 \cdot 16 \cdot 3^2 + 16) + (16 \cdot 32 \cdot 3^2 + 32) + (32 \cdot 7 \cdot 7)h + h + 10h + 10 \leq 100,000 \rightarrow h \leq 60.3$ with weight 99,550. This also applied to other configurations. As shown in Task 1-2, the Adam optimizer works the best for this FashionMNIST data set. The following cross-validates the learning rates for the 3 configurations set in Table 11.

Learning rate	1e-1	1e-2	1e-3
Adam optimizer			
Configuration 1 st	1.61/44.13%	0.22/89.9%	0.29/89.42%
Configuration 2 nd	2.41/10.02%	0.22/90.23%	0.29/89.1%
Configuration 3 rd	3.04/10%	0.26/89.72%	0.28/89.7%

TABLE 12. Training loss/Validation accuracy of Adam optimizer with different Learning rates and Kaiming (He) Uniform Initialization for each the configurations in Table 11.

Through hyperparameter tunings, Table 12 shows the highest validation accuracy is received by the second configuration in Table 11 when using Adam optimizer with learning rate = 0.01 with Kaiming (He) Uniform initialization.

Training Loss	Validation Accuracy	Testing Accuracy	Training time
0.22	90.23%	90%	10.54 minutes

TABLE 13. Training loss, Validation and Testing accuracy

The training of this model is slightly overfitting but still providing the highest validation accuracy. Using the second configuration, that use 40 hidden neurons and each 20 and 40 output channels in the first and second convolution layer with 3 kernels, Table 13 provides high validation and testing accuracy for both exceeding over 90% accuracy which is higher then the models with FCN variants (Task1-2).

Task 4:	Hidden layers neurons	Output channels in 1 st &2 nd Convolution layer	Kernel	Epoch
CNN 50K	1 21	20 & 40	3	20
CNN 20K	1 6	20 & 40	3	20
CNN 10K	1 1	20 & 40	3	20

TABLE 14. Multiple parameters/configurations of CNN model different weights.

	Training Loss 1e-2 1e-3		Validation Accuracy 1e-2 1e-3		Testing Accuracy	Weights	Training time
CNN 50K	0.2	0.33	90.5%	88.7%	91%	48,841	10.52 minutes
CNN 20K	0.56	0.51	80.54%	83.1%	89.2%	19,276	10.56 minutes
CNN 10K	2.3	1.72	10%	32%	40%	9,421	10.8 minutes

TABLE 15. Training loss, Validation/testing accuracy of Adam optimizer with different Learning rates and Kaiming (He) Uniform Initialization for each CNN model by weights.

Table 15 shows the testing classification accuracy of CNN 50K, CNN 20K, and CNN 10K models. Each model uses the same output channels in the first and second convolution layers, kernel, and epoch of the CNN 100K model from Task 3 that achieved the highest validation accuracy. But each model has different number of hidden neurons as shown in Table 14, within its weights. In Table 15, the highest validation accuracy is from the CNN 50K model with 21 hidden neurons and over 91% testing classification accuracy with learning rate=0.01. Each CNN model ran for approximately 10.5 - 11 minutes and took similar time regardless the weights. Comparing the testing classification accuracy with all CNN and FCN variants, the model with highest validation accuracy was the CNN 50K model with configurations 1 hidden layer with 21 neurons which weights 48,841, and that has output channels 20 and 40 for each the first and second convolution layer with kernel 3 and 20 epochs as presented in Tables (14, 15). Overall, CNN models performed better than FCN models because CNN detects the local patterns and spatial features learning efficiently and specifically CNN 50K model may perform better because it has lower risk of overfitting than models with 100K weight as it captures the most relevant features in a smaller network.

5. Summary and Conclusions

The goal of this study was to classify images of different fashion items from the FashionMNIST data set by applying the Fully Connected Deep Neural Networks[5] and the Convolutional Neural Networks[5] as the classifiers to categorize each image into one of the 10 clothing classes. By constructing the networks for both, hyperparameter tuning and trying different configurations was necessary to find the most accurate model for each classification to receive the highest testing accuracy that is stabled and convergent. By designing the FCN and CNN model by different weights, it allowed to compare their testing classification accuracy with all CNN and FCN variants so the most accurate model that performs with high accuracy can be selected. In this study, the model that presented highest accuracy is the CNN model with weight approximately 50K with configurations: 1 hidden layer with 21 neurons which weights 48,841, output channels 20 and 40 for each the 1st and 2nd convolution layer with kernel 3 and 20 epochs. Through hyperparameter tunings and configurations of classifiers, this study tells constructing and training the model with accurate parameters is important in classification tasks. Furthermore, this study can be extended to develop a system that classifies images of different clothing items and also other objects by using the data of 28×28 pixel for each image sample. In the future, this study could contribute valuable and enhanced resources to the IT and AI industry as the classifier algorithm would predict the category of a specific item, that could be a powerful tool for automation of sorting by categories in the fashion industry.

Acknowledgements

The author is thankful to Prof. Frank for the useful explanation about Neural Networks and cross validation as it guided the mathematical concepts and the coding layouts in the homework. The author is also thankful to the TA Rohin Gilman for going over the general information of the homework and answering specific questions for constructing the network model which helped to approach the given tasks. This report incorporates the theoretical background from the author's previous homework report [Homework 4], which expanded ideas of using similar neural networks, mathematical concepts, and different classifiers.

REFERENCES

- [1] C. da Costa-Luis, T. A. Caswell, B. Ragan-Kelley, N. Yorav-Raphael, and contributors. tqdm: A fast, extensible progress bar for python and cli. <https://github.com/tqdm/tqdm>, 2023.
 - [2] N. Frank. Amath 482 A Instructor Notes. Lecture 18-22: Neural Networks, Binary Classification, Deep Learning, and Optimization.
 - [3] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.
 - [4] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
 - [5] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in pytorch. In *NIPS-W*, 2017.
 - [6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12(Oct):2825–2830, 2011.
- [1][2][3][4][5][6]